

Part A: Analysis

1. Requirements List

General Requirements:

1. The system must allow users to report facility or IT issues on campus.
2. The system must have a web-based interface accessible via a browser.
3. The system must store data persistently (e.g., in a database).

Student Features:

4. Students must be able to log in to the system.
5. Students must be able to submit a new service request (providing a title, description, and location).
6. Students must be able to view a list of all requests they have submitted.
7. Students must be able to click on a request to view its detailed status.
8. Students cannot change the status of a request once submitted.

Staff Features:

9. Staff members must be able to log in to the system.
10. Staff must be able to view a master list of all requests submitted by all students.
11. Staff must be able to filter or sort requests by status (Pending, In Progress, Done).
12. Staff must be able to view the details of any request.
13. Staff must be able to update the status of a request from "Pending" to "In Progress" or "Done".
14. Staff must be able to add a resolution note when closing a ticket.

2. User Roles

1. **Student:** The creator of the service request. They can only view their own requests and cannot alter statuses.
2. **Staff (Admin/Facilities):** The resolver of the service request. They can view all requests and update their statuses to reflect work progress.

3. Use Cases

Use Case 1: Submit a New Request

- **Actor:** Student
- **Trigger:** The student notices a broken projector in a classroom and wants to report it.

- **Main Flow:**
 1. Student navigates to the "Create Request" page.
 2. Student fills out the form (Title: "Broken Projector", Location: "Room A201", Description: "Projector will not turn on").
 3. Student clicks "Submit".
 4. System validates the input (ensures no fields are empty).
 5. System saves the request to the database with a default status of "Pending".
- **Success Outcome:** The request is saved, and the student is redirected to their request list with a success message.

Use Case 2: View List of Personal Requests

- **Actor:** Student
- **Trigger:** The student wants to check if their previously reported issue has been fixed.
- **Main Flow:**
 1. Student navigates to the "My Requests" dashboard.
 2. System queries the database for all requests associated with the student's ID.
 3. System displays a table listing the titles, dates, and current statuses of the requests.
- **Success Outcome:** The student successfully sees a summary of all their submitted tickets.

Use Case 3: View Request Details

- **Actor:** Staff (or Student)
- **Trigger:** A staff member clicks on a specific request from the master list to see exactly what needs to be fixed.
- **Main Flow:**
 1. Staff member clicks the "View Details" button next to a request.
 2. System extracts the Request ID from the URL.
 3. System queries the database for the full details of that specific ID.

4. System renders a page showing the full description, location, submitter info, and status.
- **Success Outcome:** The user sees the complete, detailed view of the specific request.

Use Case 4: Update Request Status

- **Actor:** Staff
- **Trigger:** A staff member has finished fixing the projector and needs to close the ticket.
- **Main Flow:**
 1. Staff member views the details of the specific request.
 2. Staff member selects "Done" from a status dropdown menu.
 3. Staff member clicks "Update Status".
 4. System validates the status change.
 5. System updates the database record with the new status.
- **Success Outcome:** The request is marked as "Done", and the staff member sees the updated status on the screen.

Part B: Design

1. Class Responsibility Table (SRP)

Class Name	Responsibility (1 sentence)	Reason to change (1 reason only)
Request	Represents the data structure of a single request (id, title, status).	The database schema (data fields) for a request changes.
RequestRepository	Handles all direct database queries (SELECT, INSERT, UPDATE) for requests.	The underlying database technology or table structure changes.
RequestService	Contains business logic, such as ensuring a student cannot mark a ticket as "Done".	The business rules regarding how requests are handled change.

RequestValidator	Checks if user input (e.g., POST data) is valid before saving.	The rules for what constitutes a valid request (e.g., max title length) change.
RequestController	Receives HTTP requests, calls the Service/Repository, and passes data to the View.	The application's URL routes or user flow changes.
ViewRenderer	Loads HTML template files and injects PHP variables into them.	The way the application outputs HTML to the browser changes.

2. Simple UML Class Diagram

3. MVC Mapping Table

Item	MVC Layer	Why?
Request (Entity)	Model	It represents the core application data structure.
RequestRepository	Model	It manages reading from and writing to the data source (database).
RequestService	Model	It encapsulates the business rules and logic of the application.
RequestController	Controller	It acts as the middleman, taking user input, talking to the Model, and deciding which View to load.
requests/index.php	View	It is purely responsible for displaying the HTML list of requests to the user.
requests/show.php	View	It is purely responsible for displaying the HTML details of a single request.

4. Route Table

HTTP Method	URL Path	Controller@Action	Purpose
-------------	----------	-------------------	---------

GET	/requests	RequestController@index	Display the list of requests for the logged-in student.
GET	/requests/create	RequestController@create	Show the HTML form to submit a new request.
POST	/requests/store	RequestController@store	Process the form submission and save the new request.
GET	/requests/show	RequestController@show	View the details of a specific request (requires an ID parameter).
GET	/staff/requests	RequestController@staffIndex	Display the master list of all requests for staff members.
POST	/requests/status	RequestController@updateStatus	Process a status change submitted by a staff member.

Part C: Minimal Code Skeleton

File: app/Controllers/RequestController.php

```

class RequestController {
    // Pseudocode for handling a status update
    public function updateStatus() {
        // 1. Get the ID and new status from the POST request
        // $id = $_POST['id'];
        // $newStatus = $_POST['status'];

        // 2. Call the service to apply business logic and save
        // $requestService = new RequestService();
        // $requestService->changeStatus($id, $newStatus);

        // 3. Redirect back to the details page
        // redirect to /requests/show?id=$id
    }

    public function staffIndex() {
        // 1. Ask Repository for ALL requests
        // 2. Pass data to ViewRenderer to show staff dashboard
    }
}

```

Part D: Reflection

1. Which parts of your design are Model, View, and Controller?

In my design, the Request, RequestRepository, and RequestService make up the **Model** layer because they handle data storage, retrieval, and business rules. The .php files inside the app/Views/ folder (like index.php and show.php) are the **Views** because they contain the HTML structure. Finally, the RequestController acts as the **Controller**, receiving HTTP requests, asking the Model for data, and handing that data off to the View.

2. Where should validation happen, and why?

Validation should happen within the **Model** layer (specifically, I designed a RequestValidator class for this). It should happen *before* the Controller attempts to pass data to the Repository to be saved. If validation happens in the View (like HTML5 required tags), malicious users can bypass it. By keeping it in the Model layer, we ensure that no bad data ever enters our database, regardless of how the data was submitted.

3. What would break if you put SQL inside a View file?

If I put SQL queries (like SELECT * FROM requests) directly inside a View file like

requests/index.php, I would break the **Single Responsibility Principle (SRP)** and the core concept of MVC. If the database password changes or a table name changes, I would have to dig through HTML files to fix it. It makes the code incredibly hard to test, read, and maintain. Views should be "dumb" and only display the data they are handed.

4. What code do you expect to write next week to make this real?

Next week, I expect to write the actual public/index.php (Front Controller) and a Router class that can read my Route Table and direct traffic to the RequestController. I will also need to write the actual PDO SQL statements inside my RequestRepository to connect to a real MySQL database, and write the actual HTML/PHP foreach loops inside my View files to display the data on the screen.